



Lista 3 — React e Componentes

Aulas 21 a 30

Desenvolvimento Web

Exercícios sobre React.js, componentes, props, state, ciclos de vida e hooks. Cenários de desenvolvimento de aplicações SPA modernas.

Exercício 1

Cenário: Dashboard administrativo para startup de logística — tela de rastreamento de entregas.

1. Crie um componente Rastreador usando Vite + React que: (a) recebe um array de entregas via props, (b) renderiza cards com status (pendente, em trânsito, entregue), (c) usa cores diferentes por status.
2. Adicione um filtro por status usando estado local (useState). Os cards devem ser filtrados sem recarregar a página.
3. Implemente busca por texto: input que filtra por nome do destinatário ou código de rastreio.
4. Explique o React reconciliation: como o "key" prop ajuda o React a identificar quais elementos mudaram? O que acontece se você usar o índice do array como key?

Exercício 2

Cenário: Cliente quer um formulário multi-etapas (checkout com 3 passos) sem perder dados ao navegar entre as etapas.

1. Crie 3 componentes de etapa: Identificação (nome, email), Endereço (CEP, rua, número), Pagamento (cartão, validade, CVV).
2. Use um estado "passo" (useState) para controlar qual etapa é exibida. Os dados preenchidos devem ser preservados ao voltar.
3. Implemente validação em cada etapa: campo obrigatório, formato de email, CEP. Avance apenas se válido.
4. Extra: adicione useEffect para salvar os dados no sessionStorage quando o usuário fechar o navegador e restaurar ao retornar.

Exercício 3

Cenário: Lista infinita de produtos com scroll infinito e loading otimizado.

1. Crie um componente `ProdutosLista` que carrega 20 itens de uma API simulada (<https://jsonplaceholder.typicode.com/photos>) ao montar.
2. Implemente scroll infinito: detecte quando o usuário atinge 80% do scroll e carregue mais 10 itens. Use `IntersectionObserver` para performance.
3. Adicione estado de loading (skeleton loader), estado vazio e estado de erro com tratamento adequado.
4. Explique `useEffect` com clean-up: por que é importante retornar uma função de limpeza em efeitos com subscriptions ou timers?

Exercício 4

Cenário: Tema escuro (dark mode) que persiste preferência do usuário.

1. Implemente um `ThemeProvider` usando `React Context` que fornece `{ theme, toggleTheme }` para toda a árvore de componentes.
2. Use `localStorage` para persistir a preferência (dark ou light).
3. Aplique as cores via variáveis CSS customizadas (`--bg`, `--text`, `--primary`) e alterne a classe "dark" no `<html>`.
4. Refatore para usar `useReducer` em vez de `useState` para gerenciar o estado do tema (payload: `{type: "TOGGLE"}` ou `{type: "SET", payload: "dark"}`). Compare as duas abordagens.

Requisitos

Node.js 18+, Vite, navegador. Criar projeto: `npm create vite@latest`. JSONPlaceholder API (gratuita).